Python Tuple & Set Methods

Complete Guide with Syntax & Examples

TUPLE METHODS

A **Tuple** is an **ordered, immutable** collection. Once created, elements CANNOT be changed, added, or removed.
Example: `t = (10, 20, 30, 20, 40, 20)`

1. count()

Returns how many times a specific value appears in the tuple.

```
Syntax: tuple.count(value)
```

```
t = (10, 20, 30, 20, 40, 20)
print(t.count(20))
```

```
Output: 3
```

2. index()

Returns the INDEX of the FIRST occurrence of a value. Raises ValueError if not found.

```
Syntax: tuple.index(value, start, end) # start & end are optional
```

```
t = (10, 20, 30, 20, 40, 20)
print(t.index(30))
print(t.index(20, 2)) # search from index 2
```

```
Output: 2
3
```

Why only 2 methods? Tuples are IMMUTABLE — you cannot add, remove, or change elements after creation. So methods like append(), remove(), sort() do NOT exist for tuples. Use tuple() to convert a list to tuple, and list() to convert back.

Useful Tuple Operations (Not Methods but Important!)

len(t)

```
t = (10, 20, 30)
print(len(t))
```

```
Output: 3
```

in (membership)

```
t = (10, 20, 30)
print(20 in t)
```

```
Output: True
```

Slicing t[start:end]

```
t = (10, 20, 30, 40, 50)
print(t[1:4])
```

```
Output: (20, 30, 40)
```

Concatenation t1 + t2

```
t1 = (1, 2)
t2 = (3, 4)
print(t1 + t2)
```

```
Output: (1, 2, 3, 4)
```

Repetition t * n

```
t = (1, 2)
print(t * 3)
```

```
Output: (1, 2, 1, 2, 1, 2)
```

Convert list -> tuple

```
a = [1, 2, 3]
t = tuple(a)
print(t)
```

```
Output: (1, 2, 3)
```

SET METHODS

A **Set** is an **unordered, mutable** collection of **UNIQUE elements**. Duplicates are automatically removed.
Example: `s = {1, 2, 3, 4, 5}`

1. add()

Adds a single element to the set. If element already exists, nothing happens.

Syntax: `set.add(element)`

```
s = {1, 2, 3}
s.add(4)
print(s)
```

```
Output: {1, 2, 3, 4}
```

2. remove()

Removes a specific element. Raises KeyError if element is NOT found.

Syntax: `set.remove(element)`

```
s = {1, 2, 3, 4, 5}
s.remove(3)
print(s)
```

```
Output: {1, 2, 4, 5}
```

3. discard()

Removes a specific element. Does NOT raise error if element is not found (safer than remove).

Syntax: `set.discard(element)`

```
s = {1, 2, 3, 4, 5}
s.discard(10) # 10 not in set, no error!
print(s)
```

```
Output: {1, 2, 3, 4, 5}
```

4. pop()

Removes and returns an ARBITRARY (random) element from the set. Raises KeyError if set is empty.

Syntax: `set.pop()`

```
s = {1, 2, 3, 4, 5}
removed = s.pop()
print(removed)
print(s)
```

```
Output: 1 (any random element)
{2, 3, 4, 5} (remaining)
```

5. clear()

Removes ALL elements from the set. Makes it an empty set.

Syntax: `set.clear()`

```
s = {1, 2, 3, 4, 5}
s.clear()
print(s)
```

```
Output: set()
```

6. copy()

Returns a shallow COPY of the set. Changes to copy do not affect original.

Syntax: `new_set = set.copy()`

```
s = {1, 2, 3}
b = s.copy()
b.add(99)
print(s)
```

```
print(b)
```

```
Output: {1, 2, 3}
{1, 2, 3, 99}
```

7. union() or |

Returns a NEW set with ALL elements from BOTH sets (no duplicates).

```
Syntax: set.union(other_set) OR set1 | set2
```

```
s1 = {1, 2, 3}
s2 = {3, 4, 5}
print(s1.union(s2))
print(s1 | s2)
```

```
Output: {1, 2, 3, 4, 5}
{1, 2, 3, 4, 5}
```

8. intersection() or &

Returns a NEW set with only the COMMON elements found in BOTH sets.

```
Syntax: set.intersection(other_set) OR set1 & set2
```

```
s1 = {1, 2, 3, 4}
s2 = {3, 4, 5, 6}
print(s1.intersection(s2))
print(s1 & s2)
```

```
Output: {3, 4}
{3, 4}
```

9. difference() or -

Returns elements that are in the FIRST set but NOT in the second set.

```
Syntax: set.difference(other_set) OR set1 - set2
```

```
s1 = {1, 2, 3, 4}
s2 = {3, 4, 5, 6}
print(s1.difference(s2))
print(s1 - s2)
```

```
Output: {1, 2}
{1, 2}
```

10. symmetric_difference() or ^

Returns elements that are in EITHER set, but NOT in BOTH (opposite of intersection).

```
Syntax: set.symmetric_difference(other_set) OR set1 ^ set2
```

```
s1 = {1, 2, 3, 4}
s2 = {3, 4, 5, 6}
print(s1.symmetric_difference(s2))
print(s1 ^ s2)
```

```
Output: {1, 2, 5, 6}
{1, 2, 5, 6}
```

11. update() or |=

Adds ALL elements from another set INTO the current set (modifies in-place).

```
Syntax: set.update(other_set)
```

```
s1 = {1, 2, 3}
s1.update({4, 5})
print(s1)
```

```
Output: {1, 2, 3, 4, 5}
```

12. intersection_update()

Keeps ONLY the common elements in the current set (modifies in-place).

```
Syntax: set.intersection_update(other_set)
```

```
s1 = {1, 2, 3, 4}
s1.intersection_update({3, 4, 5})
print(s1)
```

```
Output: {3, 4}
```

13. difference_update()

Removes elements that exist in the other set from the current set (modifies in-place).

```
Syntax: set.difference_update(other_set)
```

```
s1 = {1, 2, 3, 4}
s1.difference_update({3, 4})
print(s1)
```

```
Output: {1, 2}
```

14. issubset()

Returns True if ALL elements of the current set are present in the other set.

```
Syntax: set.issubset(other_set) OR set1 <= set2
```

```
s1 = {1, 2}
s2 = {1, 2, 3, 4}
print(s1.issubset(s2))
```

```
Output: True
```

15. issuperset()

Returns True if the current set contains ALL elements of the other set.

```
Syntax: set.issuperset(other_set) OR set1 >= set2
```

```
s1 = {1, 2, 3, 4}
s2 = {1, 2}
print(s1.issuperset(s2))
```

```
Output: True
```

16. isdisjoint()

Returns True if the two sets have NO common elements (zero intersection).

```
Syntax: set.isdisjoint(other_set)
```

```
s1 = {1, 2, 3}
s2 = {4, 5, 6}
print(s1.isdisjoint(s2))
```

```
Output: True
```

17. frozenset()

Creates an IMMUTABLE version of a set. Cannot be changed after creation (like tuple for lists).

```
Syntax: fs = frozenset(iterable)
```

```
s = {1, 2, 3, 4}
fs = frozenset(s)
print(fs)
# fs.add(5) -> AttributeError! cannot modify
```

```
Output: frozenset({1, 2, 3, 4})
```

Quick Reference Summary

Method	Tuple	Set	What it does
count()	✔	✘	Count occurrences of a value
index()	✔	✘	Find position of a value
add()	✘	✔	Add one element
remove()	✘	✔	Remove by value (error if missing)
discard()	✘	✔	Remove by value (no error if missing)
pop()	✘	✔	Remove & return random element
clear()	✘	✔	Remove all elements
copy()	✘	✔	Return a copy
union() / |	✘	✔	All elements from both sets
intersection() / &	✘	✔	Common elements only
difference() / -	✘	✔	Elements in set1 not in set2
symmetric_difference() / ^	✘	✔	Elements in either but not both
update()	✘	✔	Add all elements from another set
issubset()	✘	✔	Check if subset
issuperset()	✘	✔	Check if superset
isdisjoint()	✘	✔	Check no common elements
frozenset()	✘	✔	Create immutable set

KEY DIFFERENCE: Tuple = Immutable (cannot change) | Set = Mutable + Unique elements only | List = Mutable + Allows duplicates